HeteroFormer: Heterogeneous Sequence-Feature Interaction via Generative Prototype-Conditioned Cross-Field Networks

Technical Report — Architecture & Training Methodology

May 2026

Abstract

We present **HeteroFormer**, a deep neural architecture for click-through rate (CTR) prediction that unifies *continuous sequence modeling* and *prototype-conditioned feature interaction* under a generative semantics framework. Unlike conventional approaches that treat sequence encoding and feature crossing as independent stages, HeteroFormer introduces a **Dynamic Prototype Manifold** that projects sequential user behavior into a semantic codebook space, coupled with a **Diffusion Explainer** that enforces information bottleneck constraints on the prototype-sequence joint distribution. The resulting prototype representations serve as *soft conditions* rather than hard features for a **Proto-Conditioned Cross-Field Network**, where field-wise self-attention is gently biased by prototype assignment weights and modulated by explainability gates. An **Energy Calibrator** provides post-hoc uncertainty estimation for inference-time logit calibration. The entire generative semantics layer is trained via self-supervised objectives (IB loss, reconstruction, orthogonality, and packing) and decoupled from the CTR gradient path through architectural detachment, preventing representation degeneration while regularizing the shared encoder. We describe the model architecture, mathematical formulation, training dynamics, and the MetaAligner scheduler that adaptively balances generative regularization strength against validation feedback.

Keywords: CTR Prediction, Sequence Modeling, Feature Interaction, Prototype Learning, Information Bottleneck, Cross-Field Attention, Generative Semantics

1 Introduction

Click-through rate prediction in recommendation systems demands simultaneous modeling of (i) *temporal user behavior sequences* that capture evolving interests, and (ii) *multi-field feature interactions* that encode combinatorial patterns across user profiles, item attributes, and contextual signals. Existing architectures typically address these two facets through disjoint subsystems: sequence encoders (e.g., Transformer, SSM) produce fixed-length user representations that are concatenated with static features and fed into cross-networks (e.g., DCN, xDeepFM) for explicit interaction modeling. This pipeline suffers from a fundamental representational gap: sequential dynamics are compressed into point embeddings before interaction, losing the rich uncertainty structure inherent in behavioral trajectories.

HeteroFormer bridges this gap through three architectural innovations. First, we replace the conventional *sequence-to-vector* compression with a **Dynamic Prototype Manifold** that maps variable-length sequences onto a learnable semantic codebook via differentiable optimal transport (Sinkhorn iterations). The prototype assignments yield a soft decomposition of user intent into interpretable semantic bases, akin to vector quantization but with continuous, user-conditioned rotation on the Grassmann manifold. Second, we introduce a **Diffusion Explainer** that treats the prototype-sequence joint distribution as an information bottleneck: an encoder compresses the joint representation into a low-dimensional latent space, from which a decoder reconstructs the original sequence features. The bottleneck constraint forces the prototypes to capture *minimally sufficient statistics* of user behavior, discarding noise while preserving predictive signal. Third, we design a **Proto-Conditioned Cross-Field Network** where the prototype outputs do not directly enter the prediction path; instead, they serve as *conditioning signals* that softly bias field-wise self-attention and gate feed-forward transformations. This decoupling ensures that the generative semantics layer evolves under self-supervised objectives without interfering with the discriminative CTR gradient flow.

The training protocol employs a **MetaAligner** scheduler that dynamically modulates the total weight of generative regularization based on the validation-training AUC gap, implemented as a leaky-integral PID controller. When overfitting pressure is detected, the scheduler increases auxiliary weight to strengthen regularization; during stable phases, it decays to prevent underfitting. The result is an architecture where sequence modeling and feature interaction are not merely chained, but *semantically coupled* through a shared generative substrate that provides interpretability,

uncertainty quantification, and representation regularization.

2 Architecture

The HeteroFormer architecture comprises four functional layers: (1) Foundation Encoding, (2) Generative Semantics, (3) Feature Interaction, and (4) Prediction & Calibration. Figure 1 illustrates the data flow.

Figure 1: *HeteroFormer architecture. The generative semantics layer (shaded) operates under self-supervised objectives and feeds detached conditioning signals into the proto-conditioned cross-field network. Energy calibration is applied only at inference.*

2.1 Foundation Encoding Layer

The input comprises four feature groups: user integer features u_int, item integer features i_int, user dense features u_dense, and item dense features i_dense, together with a set of sequence domains S = {seq_1, ..., seq_m}. Each sequence domain provides a variable-length list of integer feature tuples with associated timestamps.

Multi-View Encoder. Integer features are embedded via constrained L2-normalized embeddings (max-norm = 1.0) and pooled through a heterogeneous block architecture. Dense features are projected into the shared dimension d via layer-normalized linear maps. The user, item, and context representations are fused as:

$$z\_shared = LayerNorm(u + i + c), \text{ where } c = u + i$$

Task-specific view projections branch from z_shared for downstream multi-task compatibility, though the primary path uses the shared representation.

Continuous Sequence Encoder. For each domain, sequence items are embedded per-feature and concatenated into a sequence of d-dimensional vectors. We employ a **Structured State Space Model (SSM)** with learnable time-delta gating:

$$h\_t = \exp(A * delta\_t) * h\_\{t-1\} + B(x\_t),\ y\_t = C(h\_t) + D * x\_t$$

where A in R^{d x d} is initialized with negative exponentials (real spectrum), delta_t = Softplus(MLP(timestamp_t - timestamp_{t-1})) ensures continuous-time dynamics, and D is a learnable skip connection. Multiple SSM layers are stacked with residual LayerNorm. The final sequence representation is aggregated through three temporal pools: short-term (exponential decay weights), long-term (uniform average), and static (mean + std + last), producing s_short, s_long, s_static in R^d.

Multi-Sequence Fusion. When multiple sequence domains exist, domain-specific projections feed into a multi-head attention with a learned query token. The attention weights serve as domain-importance gates, yielding a fused sequence vector z_seq.

2.2 Generative Semantics Layer

This layer is the architectural centerpiece. It receives z_seq and user features u, producing interpretable semantic representations through three coupled modules. **All outputs are detached before entering the CTR path**, ensuring gradient isolation.

2.2.1 Dynamic Prototype Manifold

A learnable codebook eta in R^{K x d} stores K global prototypes, L2-normalized to the unit sphere. Given user features u, a **Cayley Rotation** generates user-conditioned prototype axes via a low-rank Lie algebra parameterization:

$$mu\_u = Normalize(eta + 0.5 * (U * diag(w(u)) * V^T * eta - V * diag(w(u)) * U^T * eta))$$

where U, V in R^{d x r} are learnable with rank r << d, and w(u) = tanh(MLP(u)) in R^r is the user-conditioned scaling. The Cayley transform preserves orthogonality and avoids explicit matrix exponentiation.

Assignment to prototypes uses a **Langevin-Sinkhorn** optimal transport solver. Cosine similarities between z_seq and rotated prototypes are sharpened by a user- and time-conditioned concentration parameter kappa:

$$\log P\_\{ij\} = kappa\_i * \tanh(cos\_sim(z\_seq,i, mu\_u,ij))$$

The Sinkhorn algorithm iteratively normalizes rows and columns of exp(-C/epsilon) with entropy regularization epsilon, yielding soft assignment matrix Pi in R^{B x K}. In training, the final iterations inject Gaussian noise (Langevin dynamics) for exploration. The prototype representation is:

$$z\_proto = Normalize(sum\_k\ Pi\_\{ik\} * mu\_u,ik)$$

2.2.2 Diffusion Explainer (Information Bottleneck)

The prototype representation and sequence features are jointly encoded into a bottleneck latent space. The encoder concatenates [z_seq; z_proto,pooled; Pi] and outputs mean mu_z and log-variance log sigma^2 of a Gaussian posterior q(z | x). Reparameterization yields latent code z = mu_z + sigma * epsilon, epsilon ~ N(0, I).

The **IB loss** enforces KL divergence to the standard prior:

$$L\_IB = -0.5 * sum\_j\ (1 + log(sigma\_j\^2) - mu\_z,j\^2 - sigma\_j\^2)$$

A decoder reconstructs z_seq from z, yielding reconstruction loss L_recon = ||Dec(z) - z_seq||^2. The explainability vector e_diff = MLP(z) is projected to R^d and serves as a field-gating signal in the cross-field network.

2.2.3 Energy Calibrator

An MLP maps [Pi; u; i] to a non-negative energy score E in R^B via Softplus output clamped to [0, 10]. During training, a **contrastive ranking loss** pulls positive-sample energies below negative-sample energies with margin m:

$$L\_energy = E\_\{(i,j)\ in\ P\ x\ N\}\ [ReLU(E\_i - E\_j + m)]$$

where P, N are positive/negative index sets, and we subsample |N| <= 5 for stability. At inference, E calibrates logits: l_cal = l - 0.1 * (E - mean(E)).

2.3 Feature Interaction Layer

This layer is the *core predictive engine*, receiving CTR gradients end-to-end. It consists of two stages: explicit bilinear crossing and proto-conditioned implicit crossing.

2.3.1 Bilinear Cross Layer

Four field vectors [u; i; c; z_seq] enter a lightweight bilinear interaction that computes upper-triangular pairwise products with learnable per-dimension weights w_{ij} in R^d:

$$f\_i\^\{out\} = f\_i + alpha * sum\_\{j>i\}\ w\_\{ij\} * f\_i * f\_j$$

where alpha = 0.1 is a residual ratio preventing interaction dominance. LayerNorm stabilizes outputs.

2.3.2 Proto-Conditioned Cross-Field Network

Field vectors are stacked into a matrix X in R^{B x F x d} (F=4 fields) with learnable field embeddings added. L layers of proto-conditioned self-attention process the fields. Each layer comprises:

(1) **Multi-head self-attention** with prototype bias: the attention logits receive a soft additive bias from the prototype assignment weights: b_proto = MLP(Pi_detach), scaled by 0.05 to avoid overwhelming the natural attention pattern.

(2) **FiLM conditioning** on sequence context: gamma-beta modulation from z_seq adapts field representations to the temporal context.

(3) **Explainability-gated FFN**: the feed-forward output is element-wise multiplied by sigmoid(MLP(e_diff,detach)), allowing the diffusion explainability to softly suppress non-informative feature dimensions.

The final field vectors are concatenated into z_final in R^{B x 4d}, serving as the predictive representation.

2.4 Prediction and Calibration Layer

The **Calibrated CTR Head** fuses static and prototype representations through a two-way gating mechanism. Context features [u; i] and prototype weights Pi predict soft gates g = Softmax(MLP([u; i; Pi])) in R^{B x 2}:

$$z\_fuse = g\_1 * MLP(z\_final) + g\_2 * (MLP(z\_proto,detach) + r\_pos)$$

where r_pos = sigma(max(Pi) - 0.5) * MLP(z_proto) is a positive-sample residual that prevents the model from collapsing to all-negative predictions when prototype activations are weak. The output logit is l = Linear(z_fuse), temperature-scaled by exp(tau) with learned tau, and calibrated at inference by the energy score as described in Section 2.2.3.

3 Mathematical Formulation

We present the complete forward pass and loss decomposition in compact form. Let theta_enc, theta_proto, theta_diff, theta_energy, theta_cross, theta_ctr denote parameter groups for encoding, prototype, diffusion, energy, cross-field, and CTR modules respectively.

3.1 Forward Pass

Given input x = (u_int, i_int, u_dense, i_dense, {s^{(m)}, l^{(m)}, t^{(m)}}_{m in S}):

```
Algorithm 1: HeteroFormer Forward Pass
─────────────────────────────────────────────────────────────────────────────────────────────
z_shared, u, i, c <- MultiViewEncoder(x; theta_enc)
z_seq <- MultiSeqFusion({SSM(s^{(m)}, l^{(m)}, t^{(m)})}_m; theta_enc)
Pi, z_proto, kappa_mean, H_assign <- DynamicPrototype(z_seq, u; theta_proto)
mu_u <- CayleyRotate(eta, u; theta_proto)     // rotated prototypes [BxKxd]
e_diff, q_diff, L_IB, L_recon <- DiffusionExplainer(z_seq, Pi, mu_u; theta_diff)
E <- EnergyCalibrator(Pi, u, i; theta_energy)
[u_c, i_c, c_c, s_c] <- BilinearCross([u, i, c, z_seq]; theta_cross)
{f_out} <- ProtoCrossField({u_c, i_c, c_c, s_c}, z_seq, Pi_detach, e_diff_detach; theta_cross)
z_final <- Concat({f_out})
l <- CalibratedCTRHead(z_final, z_proto_detach, Pi_detach, u, i; theta_ctr)
L_ortho <- |cos_sim(z_proto, e_diff)|     // prototype-diffusion orthogonality
L_pack <- PrototypePacking(eta; theta_proto)     // Section 3.2
return l, L_IB, L_recon, L_ortho, L_pack, L_energy, E, q_diff
─────────────────────────────────────────────────────────────────────────────────────────────
```

3.2 Loss Decomposition

The total loss combines discriminative CTR loss and self-supervised generative losses, weighted by the MetaAligner output lambda_aux:

$$L_{total} = L_{CTR}(l, y) + \lambda_{aux} * (\beta * L_{IB} + \gamma * L_{recon} + \delta * L_{ortho} + \zeta * L_{pack} + \eta * L_{energy})$$

where y is the binary conversion label, and {beta, gamma, delta, zeta, eta} are hyperparameters (default: 0.01, 0.05, 0.01, 0.1, 0.1 respectively). The CTR loss is Adaptive Focal Loss:

$$L_{CTR} = E[\alpha_t * (1 - p_t)^{\gamma} * BCE(l, y)] + \lambda_{\gamma} * |\gamma - \gamma_{min}|$$

with gamma = Sigmoid(g_logit) * gamma_max learned via a dedicated Adam optimizer, alpha_t = y * alpha^+ + (1-y) * alpha^- for class imbalance, and p_t = y * sigma(l) + (1-y) * (1 - sigma(l)).

Packing Loss. Prototype coherence is regularized by penalizing off-diagonal Gram matrix entries exceeding threshold tau_c:

$$L_{pack} = \sum_{i \neq j} \mathrm{ReLU}(|\eta_i^T \eta_j| - \tau_c)^2 + 0.01 * (||U^T U - I||^2 + ||V^T V - I||^2)$$

4 Training Dynamics and Optimization

4.1 Gradient Decoupling

A central design principle is **architectural gradient decoupling**. The generative semantics layer (prototype, diffusion, energy) is trained end-to-end with the CTR path via a single backward pass, but its outputs are detached (stop-gradient) before entering the prediction path. This achieves:

(1) *Representation regularization*: The shared encoder receives gradients from both L_CTR and L_gen, preventing overfitting through multi-objective pressure. (2) *Semantic preservation*: Prototypes evolve toward interpretable,

bottlenecked representations without being distorted by CTR-specific gradient biases. (3) *Stable optimization*: No alternating optimization or gradient reversal is needed; a single optimizer step updates all parameters.

Parameter groups are organized into **shared** (encoder + cross-field + CTR head), **gen** (prototype geometry + diffusion + energy), **sparse** (embeddings), and **meta** (scheduler). AdamW optimizes shared/gen with weight decay 1e-4; Adagrad handles sparse embeddings with higher learning rate.

4.2 MetaAligner Scheduler

The auxiliary weight lambda_aux is not fixed but controlled by a **leaky-integral PID controller** that observes the validation-training AUC gap:

$$g_t = \max(0, AUC_{train} - AUC_{valid}) + 0.5 * r_{CTR} + 0.3 * r_{unc}$$

where r_CTR is the CTR loss trend (increasing loss implies higher gap signal) and r_unc = max(0, mean(E) - 1.0) incorporates uncertainty growth. The PID terms are:

$$I_t = (1 - \rho) * I_{t-1} + g_t, \text{ clipped to } [-2, 2]$$

$$P = k_p * g_t, \; I = k_i * I_t, \; D = k_d * (-\text{grad } g)$$

$$\lambda_{raw} = \text{ReLU}(P + I + D), \; \lambda_{aux} = \min(\lambda_{max}, \text{EMA}(\lambda_{raw}))$$

Leak rate rho = 0.01 prevents integral windup; EMA uses fast (0.8) or slow (0.95) decay based on deviation magnitude. When validation is unavailable (inter-valid intervals), lambda_aux decays geometrically: lambda_t = 0.05 + (lambda_base - 0.05) * 0.995^{delta_t}. The default lambda_max = 0.3 caps generative influence to prevent underfitting.

4.3 Numerical Stability

Several mechanisms ensure training stability: (1) logits clamped to [-20, 20] before and after temperature scaling; (2) Sinkhorn log-probabilities clamped to [-15, 15]; (3) SSM states clamped to [-100, 100] with NaN-to-zero fallback; (4) gradient norm clipping at 1.0 for shared/gen parameters; (5) energy output clamped to [0, 10] via Softplus plus hard clamp.

5 Design Rationale and Discussion

5.1 Why Prototype-Conditioned Interaction?

Traditional feature interaction networks treat all field pairs equally, learning interaction weights from scratch for each sample. HeteroFormer instead *conditions* interaction patterns on the user's semantic prototype decomposition, effectively saying: this user is 30% interested in electronics, 50% in fashion, 20% in travel; adjust field interactions accordingly. The prototype weights Pi provide a **soft structural prior** that reduces the effective complexity of interaction learning, especially beneficial for sparse CTR data.

5.2 Why Information Bottleneck on Prototypes?

Without the bottleneck, prototypes might memorize idiosyncratic sequence noise, becoming user-specific lookup tables rather than generalizable semantic bases. The IB constraint forces prototypes to capture *compressible* structure, patterns that can be reconstructed from a low-dimensional code, naturally selecting for semantically meaningful, generalizable features. The orthogonality loss further encourages prototype diversity, preventing redundant codebook entries.

5.3 Energy as Uncertainty, Not Feature

A critical distinction from prior energy-based models: HeteroFormer's energy score E does **not** participate in training-time prediction. It is computed from detached prototypes and used solely for *post-hoc calibration* at inference. This avoids the instability of energy-based training (e.g., contrastive divergence) while retaining the benefit of uncertainty-aware confidence adjustment. High-energy samples receive conservative predictions, improving calibration without compromising ranking.

6 Conclusion

We have presented HeteroFormer, a unified architecture for CTR prediction that couples continuous sequence modeling with prototype-conditioned feature interaction through a generative semantics layer. The key technical contributions are: (1) a Dynamic Prototype Manifold with Cayley-rotated, user-conditioned codebook entries; (2) a Diffusion Explainer that enforces information bottleneck constraints on the prototype-sequence joint distribution; (3) a Proto-Conditioned Cross-Field Network where generative outputs serve as soft conditioning rather than hard features; and (4) an Energy Calibrator for inference-time uncertainty quantification. The training protocol employs architectural gradient decoupling and a PID-based MetaAligner to balance discriminative and generative objectives dynamically. This design yields an interpretable, regularized, and uncertainty-aware prediction system where sequence dynamics and feature interactions are semantically coupled rather than mechanically chained.

References

[1] Guo, H., et al. DeepFM: A factorization-machine based neural network for CTR prediction. IJCAI, 2017.

[2] Wang, R., et al. Deep & Cross Network for Ad Click Predictions. ADKDD, 2017.

[3] Lian, J., et al. xDeepFM: Combining explicit and implicit feature interactions for recommender systems. KDD, 2018.

[4] Vaswani, A., et al. Attention is all you need. NeurIPS, 2017.

[5] Gu, A., et al. Mamba: Linear-time sequence modeling with selective state spaces. ICML, 2024.

[6] Cuturi, M. Sinkhorn distances: Lightspeed computation of optimal transport. NIPS, 2013.

[7] Tishby, N., and Zaslavsky, N. Deep learning and the information bottleneck principle. IEEE ITW, 2015.

[8] Alemi, A., et al. Deep variational information bottleneck. ICLR, 2017.

[9] LeCun, Y., et al. A tutorial on energy-based learning. MIT Press, 2006.

[10] Liu, Z., et al. AutoFIS: Automatic feature interaction selection in factorization models. KDD, 2020.